

# TetherChat

---

How the Whole Project Works — a ground-up explanation from the actual source code, for interview prep.

React + Vite client · Node/Socket.IO server · Firebase & Cloudinary  
Generated 2026-08-28

# Table of Contents

1. Project summary
2. Feature index
3. Tech stack and dependencies
  - Languages & runtimes
  - Client (`client/package.json`)
  - Server (`server/package.json`)
  - Database
  - External services
4. Architecture overview
  - Request/flow example — sending a 1:1 message
  - Request/flow example — presence
5. Directory map
6. Database schema (Cloud Firestore)
  - Collection: `users` (document ID = user's `**email**`)
  - Collection: `chats` (document ID = `"_"`, the two display names sorted and joined)
  - Collection: `groups` (document ID = ``nanoid()`` group id)
  - Collection: `registered` (single document ``users_list``)
  - Relationships
  - Text ER diagram
7. Feature deep-dives
  - 7.1 Email/password sign-up + avatar upload (feature 1)
  - 7.2 Google sign-in / sign-up (feature 2)
  - 7.3 Email/password login + auth guard (features 3, 4)
  - 7.4 Presence: online users (features 5, 6, 31, 32)
  - 7.5 Friends: request / accept / decline (features 7, 8, 9, 34)
  - 7.6 Groups: create / list / open (features 10, 11, 35)
  - 7.7 Opening a 1:1 room + message persistence (features 12, 37, 38)
  - 7.8 Real-time messaging: text, emoji, stickers, dedupe (features 13, 14, 15, 33)
  - 7.9 Typing indicator (feature 16)
  - 7.10 Read receipts (feature 17)
  - 7.11 Per-chat background + bubble theme (feature 18)
  - 7.12 Scroll-to-bottom button (feature 19)
  - 7.13 Room-join request toast (feature 21)
  - 7.14 WebRTC video call (features 22, 39)
  - 7.15 Edit profile (feature 23)
  - 7.16 Settings, theme, status (features 24, 25, 26, 27)
  - 7.17 PWA install prompt + service workers (features 28, 40, 41)
8. Cross-cutting concerns
  - Authentication & authorization
  - Session / token handling
  - Middleware
  - Config & environment variables
  - Logging
  - Error handling
  - Caching (localStorage — the app's client-side cache layer)

- Background jobs

- File uploads

- Third-party API integrations

## 9. Setup and run

- Prerequisites

- Install

- Configure

- Run (two terminals)

- Build / preview / lint

- Migrate / seed

- Test

- Deploy (as configured)

## 10. Interview prep

- 10.1 The 20 most likely questions (with code-grounded answers)

- 10.2 Design decisions and rejected alternatives

- 10.3 Known weaknesses & how I'd improve/scale

- 10.4 60-second walkthrough (verbal)

- 10.5 3-minute walkthrough (verbal)

## 11. Glossary

Open questions / things I couldn't verify

# 1. Project summary

TetherChat is a real-time one-to-one and group chat web app with a companion peer-to-peer video-call screen. It is a two-part project: a **React single-page app** (in `client/`) built with Vite, and a **small Node/Express + Socket.IO server** (in `server/`) that only relays real-time events — it stores nothing in a database. All durable data (user profiles, friends, groups, and 1:1 message history) lives in **Google Firebase**: Firebase Authentication handles login/signup, and Cloud Firestore holds the documents. Users sign up with email/password or Google, upload an avatar (which is pushed to Cloudinary), and land on a Home dashboard that shows who is online, their friends, and their groups. To talk to someone you send a friend request; once accepted, either party can open a chat room, exchange text/emoji/stickers in real time, see typing indicators and read receipts, and jump into a WebRTC video call. Messages are cached in the browser's `localStorage` for instant reload and, for 1:1 chats, are also written back to Firestore when you leave the room. The app is also a PWA (installable, with a service worker and Firebase Cloud Messaging push notifications wired up). The intended user is anyone who wants a lightweight WhatsApp-style chat; in practice it's a portfolio/learning project. End to end: **authenticate** → **see online users** → **add friends** → **open a room** → **chat in real time** → **optionally video call**, with Socket.IO carrying the live traffic and Firestore/localStorage providing persistence.

# 2. Feature index

Every user-facing feature and every background/internal process, one line each.

## User-facing features

1. Email/password sign-up ( `SignUp.jsx` ) with optional profile-picture upload to Cloudinary.
2. Google sign-up/sign-in via Firebase popup ( `SignUp.jsx` , `Login.jsx` ).
3. Email/password login ( `Login.jsx` ) with friendly Firebase error messages.
4. Auto-redirect to `/home` if already authenticated; guard that kicks you back to `/` if not.
5. Home dashboard with header, online-user count, and welcome panels ( `Home.jsx` ).
6. Live "Online Users" list (header dropdown + sidebar) driven by Socket.IO ( `Home.jsx` , `Sidebar.jsx` ).
7. Send a friend request to an online user ( `Home.jsx` , `handleSendFriendRequest` ).
8. Accept / decline incoming friend requests ( `Home.jsx` , `Sidebar.jsx` ).
9. Friends list (online friends shown in Online Users, offline ones in the Friends section) ( `Sidebar.jsx` ).
10. Create a group with a name, picture, and members chosen from your friends ( `Home.jsx` , `handleCreateGroup` ).
11. Your Groups list; click to open the shared group room ( `Sidebar.jsx` , `Home.jsx` , `handleJoinGroup` ).
12. Open a 1:1 chat room with a friend / yourself ( `Home.jsx` , `handleJoinRoom` ).
13. Real-time messaging: text messages ( `Chat.jsx` , `handleSubmit` ).
14. Emoji picker ( `Chat.jsx` , `addEmoji` ).
15. Sticker packs (basic + animals) sent as sticker-type messages ( `Chat.jsx` , `sendSticker` ).
16. Typing indicator ("... is typing" bubble) ( `Chat.jsx` , `handleTyping` ).
17. Read receipts (single/double blue check based on `viewed` ) ( `Chat.jsx` ).
18. Per-chat customization: background image + message-bubble theme, persisted to localStorage ( `Chat.jsx` ).
19. "Scroll to bottom" floating button using an IntersectionObserver ( `Chat.jsx` ).
20. Message notification counts (unread badges per user) ( `Home.jsx` , `Sidebar.jsx` ).

21. Room-join request toast ("X wants you to join room ...") with Accept/Decline ( `Home.jsx` ).
22. WebRTC video call screen: start/answer call, mic toggle, camera toggle, hang up, fullscreen ( `video-call.jsx` ).
23. Edit Profile modal (display name, bio, avatar → Cloudinary) ( `Home.jsx` `handleSaveProfile` ).
24. Settings modal: dark mode, notifications toggle, privacy-mode toggle, online status, language dropdown ( `Home.jsx` ).
25. Dark/light theme toggle, persisted ( `ThemeContext.jsx` , `Home.jsx` ).
26. Set an availability status + status message broadcast to others ( `Home.jsx` `handleStatusUpdate` , `handleOnline` ).
27. Sign out (disconnect socket, Firebase signOut, clear localStorage) ( `Home.jsx` `handleSignOut` ).
28. Install-as-PWA prompt ("Install TetherChat") ( `PwaPrompt.jsx` ).
29. Animated title text effect on the header ("TetherChat") ( `BlurText.jsx` ).
30. Loading skeletons for chat and sidebar while data loads ( `ChatSkeleton.jsx` , `MessageSkeletonBubble.jsx` , `Sidebar.jsx` ).

## Background / internal processes

31. Socket.IO connection lifecycle: connect, `join` , `disconnect` , online-user roster ( `server.js` , `socketContext.js` ).
32. Server-side in-memory tracking of online users and rooms ( `server.js` ).
33. Socket rooms: join/leave a `roomId` , relay `send-message` only to that room ( `server.js` ).
34. Friend-request / friend-accepted live relay over sockets (durable copy in Firestore) ( `server.js` , `useFirestore.js` ).
35. Group create/delete live relay over sockets ( `server.js` ).
36. Firestore reads/writes for users, groups, friends, and 1:1 chat messages ( `useFirestore.js` , `useAddUser.js` , `useGetUsername.js` ).
37. localStorage caching of messages, groups, registered users, notifications, theme, and per-chat prefs.
38. Message persistence on chat-room unmount (write back to Firestore for 1:1 rooms) ( `Chat.jsx` cleanup effect).
39. WebRTC signaling relay: `offer` / `answer` / `ice-candidate` / `hangup` ( `server.js` , `video-call.jsx` ).
40. Firebase Cloud Messaging: FCM token generation + foreground `onMessage` + background service worker ( `firebase.js` , `firebase-messaging-sw.js` ).
41. PWA service worker registration and auto-update ( `main.jsx` , `vite.config.js` `VitePWA` ).
42. Click-outside handlers to close dropdowns/pickers ( `Home.jsx` , `Chat.jsx` ).
43. Responsive mobile detection hook ( `use-mobile.jsx` ).
44. Cloudinary image uploads (unsigned, `ml_default` preset) ( `SignUp.jsx` , `Home.jsx` ).

## 3. Tech stack and dependencies

### Languages & runtimes

- **JavaScript (ES modules, JSX)** — both client and server use `"type": "module"` .
- **Node.js** — runs the Socket.IO/Express server ( `server/server.js` ).

### Client ( `client/package.json` )

| Library                                                                                             | Why it's here                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>react / react-dom</b> <code>^18.3.1</code>                                                       | UI framework.                                                                                                                                                                                   |
| <b>vite</b> <code>^6.1.1</code> + <b>@vitejs/plugin-react</b>                                       | Dev server and bundler ( <code>npm run dev</code> , <code>npm run build</code> ).                                                                                                               |
| <b>react-router-dom</b> <code>^7.1.3</code>                                                         | Client-side routing ( <code>App.jsx</code> routes).                                                                                                                                             |
| <b>socket.io-client</b> <code>^4.8.1</code>                                                         | Real-time connection to the server.                                                                                                                                                             |
| <b>firebase</b> <code>^11.3.1</code>                                                                | Auth, Firestore (database), Cloud Messaging, Analytics ( <code>Firebase/firebase.js</code> ).                                                                                                   |
| <b>axios</b> <code>^1.7.9</code>                                                                    | HTTP POST for Cloudinary image uploads.                                                                                                                                                         |
| <b>cloudinary</b> <code>^2.5.1</code>                                                               | Present in deps, but uploads are actually done via a raw <code>axios</code> POST to the Cloudinary REST API — see Open Questions.                                                               |
| <b>tailwindcss</b> <code>^4.0.0</code> + <b>@tailwindcss/vite</b> + <b>tailwindcss-animate</b>      | Styling (utility classes throughout).                                                                                                                                                           |
| <b>@chakra-ui/react</b> , <b>@emotion/react</b> , <b>styled-components</b> <code>^6.1.15</code>     | UI/CSS-in-JS libs. Only <code>styled-components</code> is clearly used (the dark-mode switch <code>StyledWrapper</code> in <code>Home.jsx</code> ). Chakra appears unused — see Open Questions. |
| <b>@react-spring/web</b> <code>^9.7.5</code>                                                        | Message enter/leave animations ( <code>Chat.jsx</code> <code>useTransition</code> ) and video-call panel animations, plus <code>BlurText</code> .                                               |
| <b>gsap</b> <code>^3.12.7</code>                                                                    | Animates the video-call control bar ( <code>video-call.jsx</code> ).                                                                                                                            |
| <b>lucide-react</b> , <b>react-icons</b> , <b>react-feather</b>                                     | Icon sets.                                                                                                                                                                                      |
| <b>react-hot-toast</b> <code>^2.5.2</code>                                                          | Toast notifications.                                                                                                                                                                            |
| <b>nanoid</b> <code>^5.0.9</code>                                                                   | Random room IDs and group IDs.                                                                                                                                                                  |
| <b>@radix-ui/react-slot</b> , <b>class-variance-authority</b> , <b>clsx</b> , <b>tailwind-merge</b> | <code>shadcn/ui</code> -style <code>Button</code> component ( <code>components/ui/button.jsx</code> , <code>lib/utils.js</code> ).                                                              |
| <b>vite-plugin-pwa</b> <code>^0.21.1</code> + <b>workbox-window</b>                                 | PWA manifest, service worker, install support.                                                                                                                                                  |
| <b>next-themes</b>                                                                                  | In deps; not referenced in source (theme handled by custom <code>ThemeContext</code> ) — see Open Questions.                                                                                    |

## Server ( `server/package.json` )

| Library                              | Why it's here                                                                                    |
|--------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>express</b> <code>^4.21.2</code>  | HTTP server; only serves a <code>"Hello World!"</code> health route. The real work is Socket.IO. |
| <b>socket.io</b> <code>^4.8.1</code> | Real-time event relay (messages, presence, WebRTC signaling).                                    |
| <b>cors</b> <code>^2.8.5</code>      | Allows the browser client (different origin) to connect.                                         |
| <b>dotenv</b> <code>^16.4.7</code>   | Reads <code>PORT</code> from <code>.env</code> .                                                 |
| <b>nanoid</b> <code>^5.0.9</code>    | Imported in <code>server.js</code> but not actually used server-side — see Open Questions.       |
| <b>nodemon</b> <code>^3.1.9</code>   | Dev auto-restart ( <code>npm start</code> runs <code>nodemon server.js</code> ).                 |

## Database

- **Cloud Firestore** (NoSQL document store) is the only real database. There is **no SQL database, no ORM, and no migration files** — collections are created implicitly on first write. Schema is therefore *implied by the code*, documented in section 6.

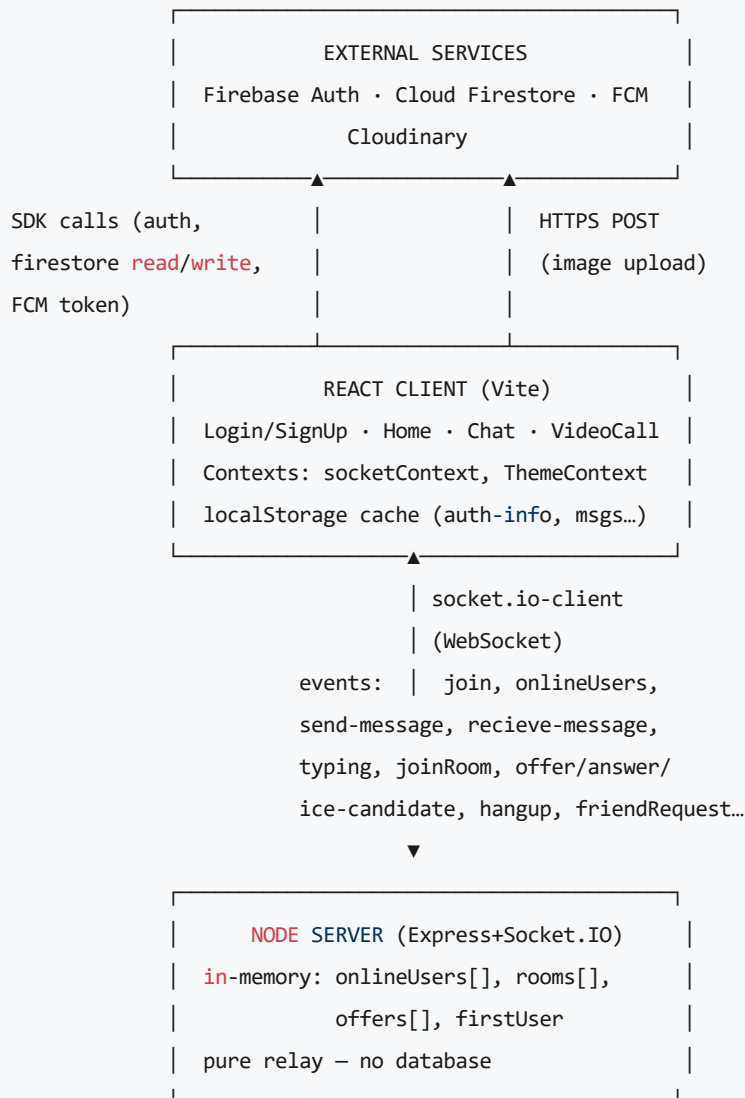
## External services

- **Firebase Auth** — identity.
  - **Cloud Firestore** — persistence.
  - **Firebase Cloud Messaging (FCM)** — push notifications.
  - **Cloudinary** — image hosting for avatars and group pictures.
  - **Render.com** — hosts the production Socket.IO server ( <https://chatapp-dcac.onrender.com> ).
- 

## 4. Architecture overview

---

Three moving parts: the **React client**, the **Socket.IO server**, and **Firebase/Cloudinary** (external managed services). The client talks to Firebase directly (SDK) for anything durable, and to the Socket.IO server for anything live. The server is deliberately dumb: it keeps a little in-memory state ( `onlineUsers` , `rooms` , `offers` ) and relays events. Nothing on the server is persisted, so a server restart forgets all presence/room state (durable data survives because it's in Firestore/localStorage).



## Request/flow example — sending a 1:1 message

User types + submits

- Chat.jsx `handleSubmit()` builds message {id,text,sender,type,viewed,timestamp}
- socket.emit("send-message", message, roomId)
  - server.js `socket.on("send-message")` → `socket.to(roomId).emit("receive-message", message)`
  - other client Chat.jsx `socket.on("receive-message")` → `setMessages([...prev, message])`
- sender also does `setMessages([...prev, message])` locally (optimistic)
- `useEffect` persists messages to `localStorage[`${roomId}`]`
- on leaving the room, cleanup effect calls `storeMessages()` → Firestore chats/<sortedPair>

## Request/flow example — presence

Home mounts → `setSocket(io(SOCKET_URL))`

- socket.emit("join", {displayName,email,profilePicUrl,status,isOnline})
- server pushes/updates `onlineUsers[]` → `io.emit("onlineUsers", onlineUsers)`
  - every client `socket.on("onlineUsers")` → `setOnlineUsers(users)`

---

## 5. Directory map

---

```

TetherChat/
├─ .gitignore           # ignores .claude and .playwright-mcp only
├─ client/              # React + Vite single-page app (the whole UI)
│  ├─ index.html        # Vite entry HTML; mounts #root, loads /src/main.jsx
│  ├─ vite.config.js     # Vite config: React plugin, Tailwind, PWA, "@"→src alias
│  ├─ vercel.json        # SPA rewrite: all paths → index.html (client-side routing)
│  ├─ components.json    # shadcn/ui config (style, aliases, Lucide icons)
│  ├─ jsconfig.json      # "@/*" path mapping for editor/IntelliSense
│  ├─ eslint.config.js   # ESLint rules
│  ├─ .env.development   # VITE_SOCKET_URL=localhost:5000 (dev)
│  ├─ .env.production    # VITE_SOCKET_URL=render URL (prod build)
│  ├─ .env.example       # template for the above
│  ├─ public/
│  │  ├─ firebase-messaging-sw.js # FCM background-message service worker
│  │  ├─ chat-192x192.png / 512 # PWA icons
│  │  └─ vite.svg        # favicon
│  └─ src/
│     ├─ main.jsx        # React root; wraps App in ThemeProvider+SocketProvider+Router; registers SW
│     ├─ App.jsx         # Route table (/ , /signup, /home, /chat/:roomId, /vc)
│     ├─ App.css         # Tailwind import + CSS variables (light/dark tokens) + keyframes
│     ├─ Firebase/
│     │  └─ firebase.js   # Firebase init: auth, db (Firestore), messaging; generateToken()
│     ├─ contexts/
│     │  ├─ socketContext.jsx # React context holding the shared socket instance
│     │  └─ ThemeContext.jsx  # Dark-mode state persisted to localStorage
│     ├─ hooks/
│     │  ├─ useFirestore.js  # ALL Firestore data access (messages, users, groups, friends)
│     │  ├─ useAddUser.js    # upsert a users/<email> document
│     │  ├─ useGetUsername.js # Look up displayName/pic by email
│     │  ├─ useGetUserInfo.js # read auth-info from localStorage
│     │  ├─ useGetRoomInfo.js # read a saved roomId for a username from localStorage
│     │  ├─ useToasts.js     # canned toast helpers (privacy/notif on/off)
│     │  └─ use-mobile.jsx   # window-width < 768 hook
│     ├─ lib/
│     │  ├─ config.js       # SOCKET_URL from VITE_SOCKET_URL (with fallback)
│     │  └─ utils.js        # cn() = clsx + tailwind-merge
│     ├─ pages/
│     │  ├─ Login.jsx       # email/password + Google login
│     │  ├─ Signup.jsx      # registration + avatar upload
│     │  ├─ Home.jsx        # dashboard: presence, friends, groups, modals, settings (~1900 lines)
│     │  ├─ Chat.jsx        # the chat room: messages, typing, stickers, themes (~930 lines)
│     │  └─ video-call.jsx  # WebRTC 1:1 video call UI + signaling (~570 lines)
│     └─ components/
│        ├─ Sidebar.jsx     # friend requests / online users / groups / friends lists
│        ├─ BlurText.jsx    # per-letter blur-in animated heading
│        ├─ SplitText.jsx   # (similar text animation component)
│        └─ PwaPrompt.jsx   # "Install TetherChat" banner

```

```

|       |— ChatSkeleton.jsx    # Loading skeleton for the message list
|       |— MessageSkeletonBubble.jsx # one skeleton bubble
|       |— ui/button.jsx      # shadcn/ui Button (cva variants)
└─ server/
    |— server.js              # Express + Socket.IO relay: presence, rooms, messages, WebRTC signaling
    |— package.json          # start = nodemon server.js
    └─ .gitignore            # node_modules, .env

```

## 6. Database schema (Cloud Firestore)

There are **no migrations or schema files** — Firestore is schemaless and collections appear on first write. The "schema" below is reconstructed from the code that reads and writes each collection. Types are JavaScript/Firestore types. Firestore auto-indexes single fields; the only query needing a composite-ish index is the `array-contains` group query (Firestore handles `array-contains` with an automatic single-field index).

### Collection: `users` (document ID = user's email)

Written by `useAddUser.js` `addUser()` (with `{ merge: true }`) and updated by the friend functions in `useFirestore.js`.

| Field                       | Type                         | Notes / default                                                                   |
|-----------------------------|------------------------------|-----------------------------------------------------------------------------------|
| <code>email</code>          | string                       | same as the doc ID                                                                |
| <code>displayName</code>    | string                       | user's name                                                                       |
| <code>profilePicUrl</code>  | string                       | Cloudinary URL, or a hardcoded default avatar URL                                 |
| <code>timestamp</code>      | Firestore<br>serverTimestamp | set on create/merge                                                               |
| <code>friends</code>        | array                        | emails of accepted friends (added by <code>acceptFriendRequest</code> )           |
| <code>friendRequests</code> | array                        | emails of people who sent ME a request (added by <code>sendFriendRequest</code> ) |
| <code>sentRequests</code>   | array                        | emails I have sent requests to                                                    |

Defined/written at: `hooks/useAddUser.js:5-14`, `hooks/useFirestore.js:146-203`. The `friends` / `friendRequests` / `sentRequests` fields are created lazily via `arrayUnion`; a brand-new user won't have them until the first friend action, so all readers default missing fields to `[]` (`getFriendData`, `useFirestore.js:189-203`).

### Collection: `chats` (document ID = `"<nameA>_<nameB>"`, the two display names sorted and joined)

Written by `storeMessages()`, read by `getMessages()` in `useFirestore.js:11-46`. The ID is built by `generateId(user1, user2) = [user1, user2].sort().join("_")`, so both participants compute the same key regardless of who opens the chat.

| Field                 | Type  | Notes                                                                                        |
|-----------------------|-------|----------------------------------------------------------------------------------------------|
| <code>messages</code> | array | the entire chat history, overwritten wholesale ( <code>setDoc(..., { merge:false })</code> ) |

Each **Message** object (built in `Chat.jsx` `handleSubmit` / `sendSticker`):

| Field                  | Type                                     | Notes                                                      |
|------------------------|------------------------------------------|------------------------------------------------------------|
| <code>id</code>        | number                                   | <code>Date.now()</code> — used as React key and dedupe key |
| <code>text</code>      | string                                   | message body or the emoji/sticker character                |
| <code>sender</code>    | string                                   | the sender's <code>displayName</code>                      |
| <code>type</code>      | string                                   | "text" or "sticker"                                        |
| <code>viewed</code>    | boolean                                  | read-receipt flag                                          |
| <code>timestamp</code> | string (ISO) for text, Date for stickers | note the inconsistency (see Open Questions)                |

### Collection: `groups` (document ID = `nanoid()` group id)

Written by `createGroup()`, read by `getUserGroups()`, deleted by `deleteGroup()` in `useFirestore.js:103-141`.

| Field                    | Type                      | Notes                                   |
|--------------------------|---------------------------|-----------------------------------------|
| <code>id</code>          | string                    | same as doc ID                          |
| <code>name</code>        | string                    | group name                              |
| <code>members</code>     | array                     | member <b>emails</b> (includes creator) |
| <code>createdBy</code>   | string                    | creator's email                         |
| <code>groupPicUrl</code> | string                    | Cloudinary URL or ""                    |
| <code>createdAt</code>   | Firestore serverTimestamp | set on create                           |

Query: `getUserGroups(email)` runs `where("members", "array-contains", email)` (`useFirestore.js:123-126`).

### Collection: `registered` (single document `users_list`)

Written by `addRegisteredUser()` in `useFirestore.js:61-96` into `registered/users_list.registeredUsers` (an array of `{email, displayName, profilePicUrl, timestamp}`). **This function is never called anywhere in the client** (see Open Questions), so this collection is effectively dead code.

## Relationships

- **User ↔ User (friendship, many-to-many)** — implemented as mirrored arrays: A's `friends` contains B's email and B's `friends` contains A's email. Pending state is split across `friendRequests` (incoming) and `sentRequests` (outgoing). "Foreign key" = the email string stored in the arrays; there is no referential integrity (Firestore doesn't enforce it).
- **User ↔ Group (membership, many-to-many)** — `groups.members` holds member emails; a user's groups are found by querying that array. No back-reference is stored on the user document.
- **User-pair ↔ Chat (one-to-one per pair)** — exactly one `chats` document exists per unordered pair of display names (the sorted-join key). The "foreign key" is the composite of the two display names embedded in the doc ID.
- **Chat ↔ Messages (one-to-many, embedded)** — messages are **not** a subcollection; they're an array field inside the single chat document, so the whole array is rewritten on save.



### 7.1 Email/password sign-up + avatar upload (feature 1)

**Key snippet (the upload → auth → persist sequence):**

```
// pages/SignUp.jsx:31-61
let profilePicUrl = "";
if (profilePic) {
  const formData = new FormData();
  formData.append("file", profilePic);
  formData.append("upload_preset", "ml_default"); // unsigned Cloudinary preset
  const response = await axios.post(
    "https://api.cloudinary.com/v1_1/dzlr1rtln/image/upload",
    formData
  );
  profilePicUrl = response.data.secure_url; // hosted image URL
}
const result = await createUserWithEmailAndPassword(auth, email, password);
const authInfo = {
  userId: result.user.uid,
  displayName: name,
  email: result.user.email,
  isAuth: true,
  profilePicUrl: profilePicUrl || "https://t3.ftcdn.net/.../default.jpg",
};
addUser({ email, name, profilePicUrl }); // upsert Firestore users/<email>
localStorage.setItem("auth-info", JSON.stringify(authInfo));
navigate("/home");
```

Line-by-line notes: the Cloudinary upload is **unsigned** — it uses the `ml_default` preset and no API secret, which is why it can run straight from the browser. `createUserWithEmailAndPassword` both creates and signs in the user (Firebase keeps its own session). The app, however, does **not** rely on Firebase's session for route guards — it invents its own `auth-info` object in `localStorage` and checks `isAuth` from there.

**Inputs / outputs / validation.** Inputs: name, email, password (all `required` on the inputs, line 145/159/173), optional image (`accept="image/*"`). Output: a Firebase user + Firestore doc + `localStorage` session. Firebase enforces password  $\geq 6$  chars and email format.

**Error cases.** `catch` maps Firebase error codes to messages (`pages/SignUp.jsx:63-74`): `auth/email-already-in-use`, `auth/invalid-email`, `auth/weak-password`, else a generic message; the message renders in a red banner (line 127).

**Edge cases / limitations.** If the Cloudinary upload throws, the whole signup aborts in the same `catch` (you can't tell a bad image from a bad password). `addUser` is fire-and-forget (not awaited), so navigation can happen before the Firestore write resolves. Note `authInfo.displayName` comes from the typed `name`, while `addUser` writes the same as `displayName`.

## 7.2 Google sign-in / sign-up (feature 2)

**What it does.** One-click auth with a Google popup; falls back to a default avatar if Google returns none.

**Trace.** `handleGoogleSignup` in both `pages/SignUp.jsx:77` and `pages/Login.jsx:74`. Both call `signInWithPopup(auth, googleProvider)` (provider built in `Firebase/firebase.js:20`), build `authInfo`, call `addUser(...)`, store to `localStorage`, and `navigate("/home")`.

```
// pages/Login.jsx:79-96
const result = await signInWithPopup(auth, googleProvider);
const photoURL = result.user.photoURL || "https://t3.ftcdn.net/.../default.jpg";
const authInfo = {
  userId: result.user.uid,
  displayName: result.user.displayName,
  email: result.user.email,
  isAuth: true,
  profilePicUrl: photoURL,
};
addUser({ email: result.user.email, name: result.user.displayName, profilePicUrl: photoURL });
localStorage.setItem("auth-info", JSON.stringify(authInfo));
navigate("/home");
```

**Error cases.** Popup errors are only `console.log`ged ( `pages/Login.jsx:98-100` ) — no user-facing message on Google failure. In `Login.jsx` a `setLoading(true)` is set but only reset via a `setTimeout(2000)` (line 102), so the spinner state and the real async flow are not tightly coupled (see Open Questions).

## 7.3 Email/password login + auth guard (features 3, 4)

**What it does.** Signs an existing user in, resolves their profile (Firestore first, then Firebase user, then email prefix), caches `auth-info`, and redirects.

**Trace.**

- `handleLogin` — `pages/Login.jsx:27`.
- `signInWithEmailAndPassword(auth, email, password)` — line 33.
- Profile lookup: `getUsername(result.user.email)` → `useGetUsername.js` runs `query(users, where("email","==",email))` and returns `{displayName, profilePicUrl}` or `{null,null}` if empty — `useGetUsername.js:6-20`.
- Fallback chain for name/pic — `pages/Login.jsx:39-44`.
- Persist + redirect — lines 52-54.

```
// pages/Login.jsx:35-53
const { displayName, profilePicUrl } = await getUsername(result.user.email);
const resolvedName = displayName || result.user.displayName || email.split("@")[0];
const resolvedPic = profilePicUrl || result.user.photoURL || "https://t3.ftcdn.net/.../default.jpg";
const authInfo = { displayName: resolvedName, userId: result.user.uid, email: result.user.email,
  profilePicUrl: resolvedPic, isAuth: true };
addUser({ email, name: resolvedName, profilePicUrl: resolvedPic });
localStorage.setItem("auth-info", JSON.stringify(authInfo));
navigate("/home");
```

**The auth guard.** There's no route-level guard component. Each protected page reads `localStorage` on mount:

- `Login.jsx:21-25` and `SignUp.jsx:19-23`: `useEffect(() => { if (isAuth) navigate("/home") }, [])` — bounce authenticated users away from auth pages.

- `Home.jsx:299-303`: `useEffect(() => { if (!isAuth) navigate("/") }, [isAuth, navigate])` — bounce unauthenticated users to login.
- `isAuth` comes from `useGetUserInfo()` which just parses `localStorage["auth-info"]` (`useGetUserInfo.js:2-11`).

**Validation / errors.** Inputs `required`. Errors mapped in `pages/Login.jsx:57-68`: `auth/user-not-found`, `auth/wrong-password`, `auth/invalid-email`, else generic; shown in a red banner. There's an artificial `await new Promise(r => setTimeout(r,1000))` before sign-in (line 32) to show the spinner.

**Limitation.** The "auth" is a localStorage boolean. Anyone can set `auth-info` manually and bypass the guard; Firestore security rules (not in this repo) would be the real gate. `Chat.jsx` / `video-call.jsx` have **no** guard at all.

## 7.4 Presence: online users (features 5, 6, 31, 32)

**What it does.** Every logged-in Home page opens a socket and announces itself with `join`; the server keeps an `onlineUsers` array and broadcasts it to everyone.

### Trace (client).

- Socket created: `Home.jsx:205-207` `useEffect(() => setSocket(io(SOCKET_URL)), [])`.
- Announce: inside the `displayName && socket` effect, `socket.emit("join", {displayName,email,profilePicUrl,status,isOnline})` — `Home.jsx:335-341`.
- Receive roster: `socket.on("onlineUsers", users => setOnlineUsers(users))` — `Home.jsx:346-348`.
- Render: header badge + dropdown (`Home.jsx:817-863`) and `Sidebar.jsx` Online Users list (`Sidebar.jsx:104-208`).

### Trace (server).

```
// server/server.js:35-49
socket.on("join", ({ displayName, email, profilePicUrl, status, isOnline }) => {
  if (onlineUsers.find((user) => user.name === displayName)) { // already online?
    onlineUsers = onlineUsers.map((user) =>
      user.name == displayName
        ? { id: socket.id, name: displayName, email, profilePicUrl, status, isOnline } // refresh socket id
        : user);
    io.emit("onlineUsers", onlineUsers);
    return;
  }
  onlineUsers.push({ id: socket.id, name: displayName, email, profilePicUrl, status, isOnline });
  io.emit("onlineUsers", onlineUsers); // broadcast to all
});
```

On `disconnect` the server removes that socket id and re-broadcasts (`server.js:149-152`).

**Edge cases / limitations.** Identity key is `displayName`, not email — two users with the same display name collide (the second overwrites the first's entry). Presence is entirely in server memory, so a Render restart drops everyone until they re-emit `join`. `isOnline` is a string `"online" / "offline"`, toggled by `handleOnline` (`Home.jsx:187-203`) and persisted to `localStorage["isOnline"]`.

## 7.5 Friends: request / accept / decline (features 7, 8, 9, 34)

**What it does.** A friendship is a mutual relationship stored in Firestore and mirrored live over sockets. You can only message or add someone to a group once you're friends.

### Trace — sending a request.

- UI: the "Add" button in `Sidebar.jsx:193-202` → `handleSendFriendRequest(user)`.
- Handler: `Home.jsx:439-457`.
  - Durable write: `sendFriendRequest(email, onlineUser.email)` → `useFirestore.js:146-157`: adds my email to their `friendRequests` and their email to my `sentRequests` (both via `arrayUnion`).
  - Optimistic UI: push to local `sentRequests`.
  - Live relay: `socket.emit("friendRequest", { from:{email,displayName,profilePicUrl}, toEmail })`.
- Server relay: `socket.on("friendRequest", p => socket.broadcast.emit("friendRequest", p))` — `server.js:66-68`.
- Recipient handling: `Home.jsx:380-386` — `socket.on("friendRequest", ({from,toEmail}) => { if (toEmail!==email) return; setFriendRequests([...]) ; toast(...) })`. The `toEmail` filter is how a broadcast is turned into a targeted message.

### Trace — accepting.

```
// hooks/useFirestore.js:160-173 (acceptFriendRequest)
await updateDoc(doc(db, "users", myEmail), {
  friends: arrayUnion(requesterEmail),
  friendRequests: arrayRemove(requesterEmail),
});
await updateDoc(doc(db, "users", requesterEmail), {
  friends: arrayUnion(myEmail),
  sentRequests: arrayRemove(myEmail),
});
```

UI wiring: `Sidebar.jsx:82-88` Accept button → `Home.jsx:459-480` `handleAcceptFriend` → durable `acceptFriendRequest`, local state move, then `socket.emit("friendAccepted", {by:{...}, toEmail})`. The requester receives it at `Home.jsx:389-399` and adds the new friend + toast.

**Decline.** `handleDeclineFriend` (`Home.jsx:482-486`) → `declineFriendRequest` (`useFirestore.js:175-186`) removes the pending entries on both sides; no socket relay (silent).

**Loading the friend graph on mount.** `Home.jsx:247-261`: `getFriendData(email)` returns `{friends, friendRequests, sentRequests}` (emails); `getUsersByEmails` (`useFirestore.js:206-226`) resolves emails to profile objects with `Promise.all` of `getDoc`s for rendering.

**Validation / edge cases.** Guards on `!email || !onlineUser?.email` prevent self/blank actions. Because live relay is a broadcast filtered by `toEmail`, an **offline** recipient simply never gets the socket event — but the durable Firestore write means they'll see the pending request next time `getFriendData` runs on login. Membership checks in the sidebar: `isFriend`, `isRequested`, `isSelf` (`Sidebar.jsx:21-24`).

## 7.6 Groups: create / list / open (features 10, 11, 35)

**What it does.** Create a named group (optional picture) whose members are chosen from your friends; it's saved to Firestore, added optimistically for the creator, and relayed live so online members get it immediately. Everyone

in a group shares one room (the group id).

**Trace — create.** `Home.jsx:595-648` `handleCreateGroup`:

```
// upload group pic (optional) to Cloudinary ... then:
const members = Array.from(new Set([...selectedUsers, email].filter(Boolean))); // emails + creator, deduped
const group = { id: nanoid(), name: groupName.trim(), members, createdBy: email, groupPicUrl };
await createGroupDoc(group); // Firestore groups/<id>
setGroups(prev => prev.some(g => g.id===group.id) ? prev : [...prev, group]); // optimistic
if (socket) socket.emit("createGroup", group); // Live relay
```

- `createGroupDoc` = `createGroup` from `useFirestore.js:103-117` (writes `groups/<id>` with `createdAt: serverTimestamp()`).
- Server: `socket.on("createGroup", g => socket.broadcast.emit("groupCreated", g))` — `server.js:56-58`.
- Recipients: `Home.jsx:367-373` — `socket.on("groupCreated", group => { if (!group.members.includes(email)) return; setGroups(...) })` (broadcast filtered by membership).

**Trace — list.** Durable load on mount: `Home.jsx:231-243` calls `getUserGroups(email)` (`useFirestore.js:120-133`, the `array-contains` query) and merges with any locally cached groups by id. Groups also persist to `localStorage["groups"]` (`Home.jsx:225-227`).

**Trace — open.** `Sidebar.jsx:220-241` group item → `handleJoinGroup(group)` (`Home.jsx:431-436`): sets `roomId = group.id`, stashes `room_<id>` in `localStorage` with `isGroup:true`, and `navigate("/chat/"+roomId, {state:{userData}})`.

**Validation.** Group name required and  $\geq 1$  member (`alert()` otherwise, `Home.jsx:596-604`). Members can only be existing friends — the member picker maps over `friends` (`Home.jsx:1503-1528`).

**Limitations.** `handleDeleteGroup` (`Home.jsx:650-653`) emits `deleteGroup` and filters local state but **does not call the Firestore** `deleteGroup`, so a "deleted" group stays in Firestore and reappears on next load. Also `handleDeleteGroup` isn't wired to any button in the current UI. Group message history is **not** written to Firestore (only 1:1 is — see 7.7); groups persist messages only in `localStorage` per room.

## 7.7 Opening a 1:1 room + message persistence (features 12, 37, 38)

**What it does.** Creates/reuses a room id for a user pair, stores room metadata so a refresh survives, and on leaving the room writes 1:1 history back to Firestore if it changed.

**Trace — open a room.** `Home.jsx:410-427` `handleJoinRoom`:

```
const ExistRoom = GetRoomInfo(user.name); // localStorage[user.name] → saved roomId or null
let roomId = ExistRoom.roomId || nanoid();
localStorage.setItem(user.name, roomId); // remember the pairing
notifications[user.name] = 0; // clear unread badge
localStorage.setItem("notifications", JSON.stringify(notifications));
setJoinInfo({ from: displayName, roomId });
localStorage.setItem(`room_${roomId}`, JSON.stringify(user)); // survive refresh
navigate(`/chat/${roomId}`, { state: { userData: user } });
socket.emit("requestJoin", { from: displayName, to: user.id, roomId }); // ping the other user
```

`requestJoin` is relayed to the target socket ( `server.js:51-53` ) and shows the join-request toast on the other Home page ( `Home.jsx:361-365` , rendered at `1553-1593` ).

**Trace — load messages (Chat mount).** `Chat.jsx:244-276` : read `localStorage["messages_"+roomId]` first; if absent, fall back to Firestore `getMessages(displayName, userData?.name)` ( `useFirestore.js:27-46` , keyed on the sorted display-name pair), then cache to localStorage. `msgLen_<roomId>` records the DB length for change detection.

**Trace — persist on unmount.** `Chat.jsx:297-334` cleanup:

```
const dbLen = JSON.parse(localStorage.getItem(`msgLen_${roomId}`));
if (!isGroup && messagesRef.current.length > 0) {
  if (dbLen !== messagesRef.current.length) { // only if the count changed
    if (senderRef.current == null) senderRef.current = userData?.name;
    if (senderRef.current) {
      storeMessages(displayName, senderRef.current, messagesRef.current); // Firestore setDoc(merge:false)
      localStorage.setItem(`msgLen_${roomId}`, JSON.stringify(messagesRef.current.length));
    }
  }
}
socket.disconnect();
```

`senderRef` / `messagesRef` are refs kept in sync by effects ( `Chat.jsx:287-295` ) so the cleanup closure sees the latest values, not stale ones.

**Edge cases / limitations.** Change detection is by array **length** only — editing a message in place (same length) wouldn't persist (there's no edit feature, so this is fine today). Group rooms deliberately skip Firestore persistence ( `!isGroup` ). Because `storeMessages` does `merge:false` , it overwrites the entire history each time — last writer wins, so simultaneous writes from both participants can clobber each other.

## 7.8 Real-time messaging: text, emoji, stickers, dedupe (features 13, 14, 15, 33)

**What it does.** Sends a message object to the room over sockets; both sides append it to state, dedupe by `id` , animate it in, and cache it.

**Trace — send.** `Chat.jsx:455-483` `handleSubmit` :

```
socket.emit("typing", false, roomId);
if (newMessage.trim() === "") return;
const message = { id: Date.now(), text: newMessage, sender: displayName,
                  type: "text", viewed: false, timestamp: new Date().toISOString() };
if (inRoom.length === 2) { message.viewed = true; } // both present → read
else if (inRoom.length === 1) {
  if (userData?.name == displayName) message.viewed = true; // chatting with yourself
  else socket.emit("get-user-notif", userData?.name, message); // recipient offline-from-room → notify
}
socket.emit("send-message", message, roomId);
setMessages(prev => [...prev, message]); // optimistic local add
setNewMessage("");
```

- Server: `socket.on("send-message", (message, roomId) => socket.to(roomId).emit("recieve-message", message))` — `server.js:145-147`.
- Receive: `Chat.jsx:381-387` — appends unless an item with the same `id` already exists (dedupe).
- Render: `useTransition` over `messages` (`Chat.jsx:180-189`) animates each bubble; sent vs received styling from `send / recieve` theme objects (`Chat.jsx:728-776`).

**Emoji.** `addEmoji(emoji)` appends to the input (`Chat.jsx:485-488`); picker grid at `829-846`. **Stickers.** `sendSticker` builds a `type:"sticker"` message and emits it (`Chat.jsx:490-504`); sticker bubbles render larger (`text-4x1`, line 745-748). Sticker `timestamp` is a `Date` (not ISO string) — a minor inconsistency.

**Notification path for a not-in-room recipient.** `get-user-notif` → server looks up the online user and echoes `user-notif` back to the sender (`server.js:133-135`); the sender then emits `message-notif` to that user's socket (`Chat.jsx:373-379`, server relay `server.js:137-139`); the recipient's Home stores the message and bumps the unread count (`Home.jsx:350-359`).

**Edge cases.** `id = Date.now()` can collide if two messages are sent in the same millisecond; dedupe would then wrongly drop one. Message ordering relies on socket delivery order (no server sort).

## 7.9 Typing indicator (feature 16)

**Trace.** On input change, `handleTyping` (`Chat.jsx:191-206`) emits `socket.emit("typing", true, roomId)` once; when the field is emptied it emits `typing,false` after 500 ms. Server relays to the room: `socket.on("typing", (state, roomId) => socket.to(roomId).emit("IsSenderTyping", state))` (`server.js:163-165`). Receiver sets `IsSenderTyping` (`Chat.jsx:389-395`) and shows a three-dot bounce bubble (`Chat.jsx:777-798`).

**Limitation.** There's no debounce/timeout to auto-clear "typing" while the user keeps typing without emptying the box — it only turns off on submit or when the input becomes empty. So a sender who types and walks away can leave the indicator on until they send.

## 7.10 Read receipts (feature 17)

`viewed` is set at send time based on how many people are in the room (`Chat.jsx:469-477`, above). The double-check icon renders blue when `message.viewed` is true, grey otherwise, and only on the sender's own messages (`Chat.jsx:762-772`). `handleViewMessages` (`Chat.jsx:226-242`) exists to flip incoming messages to `viewed:true` but is **commented out at its call site** (`Chat.jsx:410`), so received messages aren't actively marked viewed after the fact — see Open Questions.

## 7.11 Per-chat background + bubble theme (feature 18)

Two preset lists: `backgrounds` (`Chat.jsx:48-73`) and `bubbleThemes` (`Chat.jsx:23-46`). Selecting a background sets `backdrop` and writes `localStorage["prefBackdrop"]` (`Chat.jsx:597-601`). Selecting a bubble theme calls `setBubbleTheme(sent, received)` which stores `prefSend / prefRecieve` (`Chat.jsx:174-179`). All three are initialized from `localStorage` on mount (`Chat.jsx:146-163`) so preferences persist across sessions. The background is applied as an inline `backgroundImage` on the messages container (`Chat.jsx:681-689`).

## 7.12 Scroll-to-bottom button (feature 19)

An `IntersectionObserver` watches a sentinel `<div ref={messagesEndRef}>` at the end of the list; when it's not visible (you've scrolled up), `showScrollButton` becomes true ( `Chat.jsx:209-224` ). Clicking the floating button calls `scrollToBottom()` → `messagesEndRef.current.scrollIntoView({behavior:"smooth"})` ( `Chat.jsx:442-449` ). The list also auto-scrolls on new messages / typing changes ( `Chat.jsx:451-453` ).

## 7.13 Room-join request toast (feature 21)

When you open a chat with someone, `requestJoin` pings their socket; their Home shows a slide-in card ( `Home.jsx:1553-1593` ) with the requester and room id. Accept → `handleAccept` navigates them into `/chat/<roomId>` ( `Home.jsx:584-587` ); Decline → hides it ( `Home.jsx:589-591` ). Visibility animates via the `isVisible` state and a `translate-x` class.

## 7.14 WebRTC video call (features 22, 39)

**What it does.** A peer-to-peer audio/video call using the browser's WebRTC APIs, with the Socket.IO server acting purely as the signaling channel (exchanging SDP offers/answers and ICE candidates).

**Entry.** From a chat, the video button `navigate("/vc", { state: { userData, roomId } })` ( `Chat.jsx:563` ). The `/vc` route renders `video-call.jsx` ( `App.jsx:19` ). Note: `video-call.jsx` never reads that navigation state — the call is **not** scoped to the room (see Open Questions).

**Socket lifecycle.** Created once via a ref guard and torn down on unmount ( `video-call.jsx:21-30` ):

```
const socketRef = useRef(null)
if (socketRef.current === null) socketRef.current = io(SOCKET_URL)
const socket = socketRef.current
useEffect(() => () => socketRef.current?.disconnect(), [])
```

**Local media.** On mount, `getUserMedia({video,audio})` fills the local `<video>`; if the camera fails it retries audio-only and flips `isVideoOn` off ( `video-call.jsx:135-169` ).

**Starting a call (caller).** `startCall` ( `video-call.jsx:323-333` ):

```
await createPeerConnection()           // new RTCPeerConnection with STUN + a placeholder TURN
await getUserMedia()                   // addTrack() each local track
const offer = await peerConnection.current.createOffer()
await peerConnection.current.setLocalDescription(offer)
socket.emit("offer", offer)            // → server broadcasts to everyone
```

`createPeerConnection` ( `video-call.jsx:276-304` ) sets `onicecandidate` (emit `ice-candidate`) and `ontrack` (attach remote stream to the remote `<video>`).

**Answering (callee).** The server stores offers and pushes them to newcomers ( `server.js:169-184` ). On `socket.on("offer")` the client calls `createAnsElems(offer)` which un-hides the Answer button and wires its click to `answerCall(offer)` ( `video-call.jsx:369-375` ). `answerCall` ( `video-call.jsx:260-274` ) creates the peer connection **with** the remote offer, gets media, creates an answer, sets local description, and emits `answer`. The caller receives `answer` and `setRemoteDescription` ( `video-call.jsx:88-93` ).

**ICE.** Candidates are relayed both ways ( `server.js:190-192` ); the receiver adds them, with a small retry if the remote description isn't set yet ( `video-call.jsx:95-112` ).

**Hang up.** `endCall` closes the peer connection, stops tracks, restarts the local preview, and emits `hangup` ( `video-call.jsx:335-367` ); server broadcasts `hangup` and clears stored offers ( `server.js:195-199` ); the other side runs `endCall` ( `video-call.jsx:114-124` ).

**Controls & animation.** Mic/camera toggles enable/disable the corresponding tracks ( `video-call.jsx:244-258` ). `@react-spring/web` animates the local/remote panels; `gsap` slides the control bar in ( `video-call.jsx:208-222` ). Fullscreen uses the Fullscreen API ( `video-call.jsx:234-242` ).

**Limitations / edge cases (significant).** Signaling is **global broadcast**, not room-scoped:

`socket.broadcast.emit("offer", ...)` sends to every connected socket, and the server also replays `offers[0]` to any newly-connecting socket ( `server.js:170-172` ). With more than two users online this breaks down — a third user could receive/answer a call meant for someone else. The TURN server is a placeholder ( `turn:your-turn-server.com` with dummy creds, `video-call.jsx:286` ), so calls that need TURN (strict NATs) will fail; only the Google STUN server actually works. There's a `remote video placeholder` canvas that paints "Remote User" until a real track arrives ( `video-call.jsx:172-205` ).

## 7.15 Edit profile (feature 23)

`openProfileModal` seeds the edit form ( `Home.jsx:500-507` ). `handleSaveProfile` ( `Home.jsx:523-579` ): validates non-empty name; if a new avatar was chosen, uploads to Cloudinary; updates `auth-info` in `localStorage` and the `profile` state; and re-emits `join` so other online users see the new name/avatar. Errors are toasted. **Note:** it updates `localStorage` but does **not** write the new name/bio back to the Firestore `users` doc — so the durable profile can drift from the local one (bio isn't stored in Firestore at all).

## 7.16 Settings, theme, status (features 24, 25, 26, 27)

- **Dark mode:** `ThemeContext` initializes from `localStorage["theme"]` (handling both JSON-boolean and legacy `"dark"` string), persists on change ( `ThemeContext.jsx:8-23` ), and `Home.jsx:305-312` toggles the `dark` class on `<html>`. The styled-components switch is `StyledWrapper` ( `Home.jsx:1833-1893` ).
- **Notifications / Privacy toggles:** local booleans with canned toasts ( `Home.jsx:144-172` , toasts in `useToasts.js` ). These are cosmetic — they don't gate any real behavior.
- **Online status:** `handleOnline` flips `isOnline` and re-emits `join` , persisting to `localStorage` ( `Home.jsx:187-203` ); `handleStatusUpdate` broadcasts a free-text status ( `Home.jsx:175-185` ).
- **Sign out:** `handleSignOut` ( `Home.jsx:488-497` ) disconnects the socket, `signOut(auth)` , `localStorage.clear()` , navigate to `/` .

## 7.17 PWA install prompt + service workers (features 28, 40, 41)

- **Install prompt:** `PwaPrompt.jsx` listens for `beforeinstallprompt` , shows a banner, and calls `deferredPrompt.prompt()` on click; "Maybe later" records a 24-hour dismissal in `localStorage` ( `PwaPrompt.jsx:51-61` ).
- **App service worker:** `VitePWA({ registerType:"autoUpdate", devOptions:{enabled:true}, manifest:{...} })` in `vite.config.js` ; registered in `main.jsx:6-8` via `registerSW({ immediate:true })` .

- **FCM:** `generateToken()` (`firebase.js:25-40`) requests notification permission and fetches an FCM token with a VAPID key; foreground messages log via `onMessage` (`Home.jsx:209-214`). Background pushes are handled by `public/firebase-messaging-sw.js` which shows a system notification. Both are called on Home mount. (In practice these mostly `console.log`; there's no server pushing FCM messages in this repo.)

## 8. Cross-cutting concerns

### Authentication & authorization

- **Provider:** Firebase Auth (email/password + Google). Set up in `Firebase/firebase.js:2-4,20-21`.
- **Session model:** the app's own `auth-info` object in `localStorage` is the source of truth for the UI, **not** Firebase's session. Written on login/signup, read by `useGetUserInfo()` (`hooks/useGetUserInfo.js`).
- **Route protection:** per-page `useEffect` redirects (see 7.3). No `<ProtectedRoute>` wrapper; `/chat/:roomId` and `/vc` are unguarded.
- **Authorization:** none enforced client-side beyond the friends gate for messaging/groups. There are no Firestore security rules in the repo, so authorization depends entirely on rules configured in the Firebase console (out of scope here).

### Session / token handling

- Firebase manages its own auth token internally (refreshes silently). The app doesn't read or attach it anywhere.
- **FCM token:** fetched in `generateToken()` with a hardcoded VAPID key (`firebase.js:32-35`); currently only logged.
- The socket carries no auth token — any client can connect and emit any event.

### Middleware

- **Server:** `app.use(cors())` (`server.js:26`) and Socket.IO CORS `origin:"*"` (`server.js:19-24`). No auth middleware, no rate limiting, no validation middleware.
- **Client:** React Router is the only "middleware-like" layer; `vercel.json` rewrites all paths to `index.html` so deep links (e.g. `/chat/abc`) work on refresh.

### Config & environment variables

- **Client:** `VITE_SOCKET_URL` selects the backend; resolved in `lib/config.js` with a hardcoded Render fallback. Files: `.env.development`, `.env.production`, `.env.example`. Only `VITE_`-prefixed vars are exposed to the browser (Vite rule).
- **Server:** `PORT` from `.env` via `dotenv` (`server.js:9`, `208`); defaults to 5000.
- **Hardcoded config (not env):** the entire Firebase config and API key (`firebase.js:8-16`, also duplicated in `firebase-messaging-sw.js`), the Cloudinary cloud name `dzlr1rt1n` and preset `m1_default`, the FCM VAPID key, and the default avatar URL. These are committed to the repo.

### Logging

- Server: `console.log` on connect/disconnect/join/hangup (`server.js`). No structured logging.
- Client: extensive `console.log` / `console.error` throughout (message loads, user loads, errors). No client error reporting service.

## Error handling

- Firebase calls in hooks are wrapped in `try/catch` that log and return safe defaults (`[]` or null-ish objects) — e.g. `getMessages`, `getFriendData`, `getUsersByEmails`. This "fail soft" style means the UI rarely crashes but errors can pass silently.
- Auth pages map error codes to user-facing banners (see 7.1, 7.3).
- WebRTC and media errors are caught and often downgraded (audio-only fallback) or just logged.

## Caching (localStorage — the app's client-side cache layer)

| Key                                                                          | Written by                       | Purpose                                              |
|------------------------------------------------------------------------------|----------------------------------|------------------------------------------------------|
| <code>auth-info</code>                                                       | Login/SignUp/profile save        | the session object (identity + <code>isAuth</code> ) |
| <code>messages_&lt;roomId&gt;</code>                                         | Chat effects                     | cached message history per room                      |
| <code>msgLen_&lt;roomId&gt;</code>                                           | Chat                             | last-persisted length, for change detection          |
| <code>room_&lt;roomId&gt;</code>                                             | Home join handlers               | room metadata to survive refresh                     |
| <code>&lt;username&gt;</code>                                                | Home <code>handleJoinRoom</code> | remembered roomId for a user pair                    |
| <code>groups</code>                                                          | Home                             | cached group list                                    |
| <code>registeredUsers</code>                                                 | Home                             | cached user directory from Firestore                 |
| <code>notifications</code>                                                   | Home                             | per-user unread counts                               |
| <code>theme</code>                                                           | ThemeContext / Home              | dark-mode boolean                                    |
| <code>isOnline</code>                                                        | Home                             | availability string                                  |
| <code>prefSend</code> / <code>prefRecieve</code> / <code>prefBackdrop</code> | Chat                             | chat appearance prefs                                |
| <code>pwaPromptDismissed</code>                                              | PwaPrompt                        | 24h snooze timestamp                                 |

## Background jobs

- None on the server (it's event-driven only). The only "background" work is the service workers (PWA caching + FCM background messages) and the in-memory presence/room bookkeeping.

## File uploads

- Avatars and group pictures upload directly from the browser to Cloudinary via unsigned `axios.post` with the `ml_default` preset (`SignUp.jsx:38`, `Home.jsx:537 / 613`). No server involvement; the returned `secure_url` is stored in Firestore/localStorage.

## Third-party API integrations

- **Firestore** (Auth, Firestore, Messaging, Analytics) — `firebase.js`.
- **Cloudinary** — image hosting (REST upload).
- **Socket.IO server on Render** — real-time relay.
- **Google STUN** (`stun:stun.l.google.com:19302`) — WebRTC NAT traversal.

## 9. Setup and run

## Prerequisites

- Node.js (v18+; the machine used v24). npm.
- A Firebase project (Auth + Firestore enabled) if you want auth/persistence to work against your own backend. The committed config points at the existing `connect1y-9d39a` project.
- A Cloudinary account with an unsigned upload preset if you want your own image hosting.

## Install

```
# from the repo root
cd server && npm install
cd ../client && npm install
```

## Configure

- **Client:** `client/.env.development` already sets `VITE_SOCKET_URL=http://localhost:5000`. Copy `.env.example` to `.env.local` to override per machine. For production, `client/.env.production` points at the Render URL.
- **Server:** optionally create `server/.env` with `PORT=5000`.
- **Firebase/Cloudinary:** to use your own, edit `client/src/Firebase/firebase.js` (and `client/public/firebase-messaging-sw.js`) and the Cloudinary cloud name/preset in `SignUp.jsx` and `Home.jsx`.

## Run (two terminals)

```
# terminal 1 - the realtime server (nodemon)
cd server && npm start           # → http://localhost:5000 ("Hello World!" on GET /)

# terminal 2 - the client (Vite dev server)
cd client && npm run dev         # → http://localhost:5173 by default
```

Open the client URL, sign up, and open a second browser/incognito window to chat between two accounts.

## Build / preview / lint

```
cd client
npm run build      # production build (uses .env.production) → dist/
npm run preview    # serve the built app locally
npm run lint       # eslint
```

## Migrate / seed

- **No migrations** (Firestore is schemaless; collections appear on first write).
- **No seed script.** Data is created by using the app (signing up creates `users` docs, etc.).

## Test

- **No tests exist.** `server/package.json` `test` script is the default `echo "Error: no test specified" && exit 1`. The client has no test tooling configured.

## Deploy (as configured)

- **Client:** Vercel ( `vercel.json` SPA rewrite present).
- **Server:** Render (the production socket URL is a Render app).

## 10. Interview prep

### 10.1 The 20 most likely questions (with code-grounded answers)

1. **What is TetherChat, in one sentence?** A real-time chat + WebRTC video app: a React/Vite front end, a thin Socket.IO relay server, and Firebase (Auth + Firestore) for persistence, with Cloudinary for images.
2. **Why Socket.IO and Firebase — isn't that redundant?** They do different jobs. Socket.IO carries **ephemeral live** events (presence, in-flight messages, typing, WebRTC signaling) with low latency; Firestore is the **durable** store (users, friends, groups, 1:1 history). The server stays stateless-ish and cheap because it never touches a DB. Trade-off: two sources of truth that must be reconciled (I do that with `toEmail` -filtered broadcasts + durable writes, e.g. friends in `Home.jsx:380-399` + `useFirestore.js:146-173`).
3. **How does presence work and what's its weakness?** Clients emit `join`; the server keeps an in-memory `onlineUsers` array keyed by **displayName** and broadcasts it ( `server.js:35-49` ). Weaknesses: identity by display name (collisions), and everything is lost on server restart. I'd key by uid and persist presence in Redis for horizontal scaling.
4. **How is a chat room identified?** Two ways. The URL `roomId` is a `nanoid` (or the group id) used for Socket.IO rooms and localStorage. The **durable** 1:1 key is `[nameA,nameB].sort().join("_")` ( `useFirestore.js:6-8` ) so both users deterministically resolve the same Firestore doc.
5. **Walk me through sending a message.** `handleSubmit` builds `{id,text,sender,type,viewed,timestamp}`, sets `viewed` from how many people are in the room, emits `send-message` to the room, and optimistically appends locally ( `Chat.jsx:455-483` ). The server relays `recieve-message` to the room ( `server.js:145-147` ); the peer appends with an id-dedupe ( `Chat.jsx:381-387` ).
6. **How do messages persist? Why only on unmount?** Live messages cache to `localStorage["messages_<roomId>"]` on every change. On leaving the room, the cleanup effect writes the whole array to Firestore *only if the length changed* and only for 1:1 rooms ( `Chat.jsx:297-334` ). Writing on unmount batches the whole conversation into one `setDoc` instead of a write per message — cheaper on Firestore, at the cost of losing unsynced messages if the tab is killed hard.
7. **Why `setDoc(..., {merge:false})` for messages?** The messages array is stored as one field; I replace it wholesale so deletes/edits would propagate. Downside: last-writer-wins, so concurrent writes from both participants can overwrite each other. A subcollection with per-message docs would fix that.
8. **How do friend requests reach an offline user?** The durable write ( `sendFriendRequest` ) adds the pending email to the recipient's `friendRequests` array immediately; the socket relay is just for instant UX when they're online ( `server.js:66-68`, filtered by `toEmail` in `Home.jsx:380-386` ). On next login, `getFriendData` loads the pending request from Firestore.
9. **How does the WebRTC call work?** Standard offer/answer/ICE over Socket.IO signaling. Caller: `getUserMedia` → `createOffer` → `setLocalDescription` → emit `offer`; callee answers symmetrically; ICE candidates relayed both ways; `ontrack` attaches the remote stream ( `video-call.jsx:260-333`, `server.js:181-199` ).

10. **What's broken or risky about the video call?** Signaling is broadcast to *all* sockets, not the room, and the server replays the first stored offer to every new connection ( `server.js:170-172,181-184` ). It only works reliably with exactly two people online. The TURN server is a placeholder, so restrictive NATs fail. I'd scope signaling to the `roomId` and add a real TURN server.
11. **How is auth enforced?** Firebase does the actual authentication; the UI gates on a `localStorage` `auth-info.isAuth` flag checked in page `useEffects` ( `Home.jsx:299-303` ). This is UX-only — real security must be Firestore security rules server-side. `/chat` and `/vc` aren't even gated.
12. **Where are secrets? Isn't the Firebase API key exposed?** The Firebase "apiKey" is in `firebase.js` and is a public client identifier (safe to ship) — the real protection is Firestore/Storage rules and Auth settings. The Cloudinary upload is unsigned by design. There are no server secrets in client code. That said, committing them is a smell; I'd move them to env vars.
13. **Why localStorage instead of Context/Redux for auth?** It survives reloads with zero setup and the app is small. `useGetUserInfo` reads it synchronously so pages can guard on mount. Trade-off: no reactivity (changes don't broadcast) and it's spoofable.
14. **How does dark mode persist and avoid a flash?** `ThemeContext` reads `localStorage["theme"]` in the `useState` initializer (so it's correct on first render) and toggles the `dark` class on `<html>` ( `ThemeContext.jsx` , `Home.jsx:305-312` ). It even handles a legacy `"dark"` string format for backward compatibility.
15. **What's the PWA story?** `vite-plugin-pwa` with `autoUpdate` , a manifest, and icons ( `vite.config.js` ); `registerSW({immediate:true})` in `main.jsx` ; a custom install banner ( `PwaPrompt.jsx` ); and a separate FCM background service worker ( `firebase-messaging-sw.js` ).
16. **How do you prevent duplicate messages?** Receiver checks `prev.some(m => m.id === message.id)` before appending ( `Chat.jsx:381-387` ). Since the sender also adds optimistically and echoes go only to *other* sockets ( `socket.to(roomId)` ), the sender won't get its own message back.
17. **How are images handled?** Uploaded straight to Cloudinary from the browser (unsigned preset), and only the resulting URL is stored ( `SignUp.jsx:38-42` , `Home.jsx:533-542` ). Keeps large blobs out of Firestore and off my server.
18. **How would you scale the server?** It's currently single-instance in-memory. I'd move presence/rooms to Redis and use the Socket.IO Redis adapter so multiple instances share state, then put them behind a load balancer with sticky sessions.
19. **What happens on a hard refresh inside a chat?** React Router `location.state` is gone, so `Chat.jsx:140-142` falls back to `localStorage["room_<roomId>"]` for the peer/group metadata, and messages reload from `localStorage` or Firestore.
20. **Biggest thing you'd refactor?** `Home.jsx` is ~1,900 lines mixing presence, friends, groups, modals, and settings. I'd split it into feature components/hooks ( `useFriends` , `useGroups` , `usePresence` ) and add Firestore security rules + a real auth guard.

## 10.2 Design decisions and rejected alternatives

| Decision                                          | Alternative rejected                                       | Trade-off                                                                                                        |
|---------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Socket.IO relay + Firebase persistence            | All-Firebase (Firestore realtime listeners for messages)   | Sockets give lower latency and free WebRTC signaling; but I now maintain two data paths and must reconcile them. |
| Stateless relay server (in-memory only)           | Server-side DB/session store                               | Simple and cheap; but presence/rooms vanish on restart and it can't scale horizontally without Redis.            |
| localStorage <code>auth-info</code> as UI session | Firebase <code>onAuthStateChanged</code> + Context         | Instant, synchronous guards and offline-friendly; but not reactive and spoofable.                                |
| Messages as one array field, saved on unmount     | Per-message subcollection docs written live                | Fewer writes, cheaper; but last-writer-wins and lost-on-crash risk.                                              |
| Friendship as mirrored arrays on user docs        | A dedicated <code>friendships</code> collection with edges | Trivial reads ( <code>array-contains</code> ); but no integrity and array size limits at scale.                  |
| Unsigned client→Cloudinary upload                 | Upload through my server                                   | No backend load, simplest; but the preset is public and abusable.                                                |
| Custom <code>ThemeContext</code>                  | <code>next-themes</code> (in deps)                         | Full control; but I shipped an unused dependency.                                                                |

### 10.3 Known weaknesses & how I'd improve/scale

- **Security:** localStorage-only guard, unguarded `/chat` & `/vc`, no Firestore rules in repo, committed keys. → Add security rules, a real `<ProtectedRoute>`, and env-based config.
- **Video call correctness:** broadcast signaling breaks with >2 users; placeholder TURN. → Room-scoped signaling + real TURN (coturn/Twilio).
- **Presence identity by displayName:** collisions. → Key by uid/email.
- **Message integrity:** whole-array overwrite, length-only change detection, `Date.now()` id collisions. → Subcollection + UUIDs + server timestamps.
- **Scale:** single in-memory server. → Redis adapter + multiple instances.
- **Code health:** giant `Home.jsx`, unused deps (Chakra, next-themes, cloudinary SDK), dead code (`addRegisteredUser`, `handleViewMessages` call commented out, unused `deleteGroup` wiring). → Split components, prune deps, remove dead code, add tests (there are none).
- **Profile drift:** profile edits update localStorage but not the Firestore user doc.

### 10.4 60-second walkthrough (verbal)

"TetherChat is a real-time chat and video app. The front end is React with Vite; the back end is a small Node + Socket.IO server that just relays live events and stores nothing. Everything durable — accounts, friends, groups, and message history — lives in Firebase: Firebase Auth for login, Firestore for the documents, and Cloudinary for profile images. You sign up, see who's online in real time over a WebSocket, send a friend request, and once it's accepted you open a chat room. Messages, typing indicators, and read receipts all flow through Socket.IO rooms, and I cache them in localStorage for instant reloads, syncing one-to-one history back to Firestore when you leave the room. There's also a WebRTC video call that uses the same server for signaling. It's a PWA, so it's installable with push notifications wired up. The interesting design tension is having two data paths — live sockets and durable Firestore — and reconciling them."

### 10.5 3-minute walkthrough (verbal)

"Let me go end to end. **Auth:** the entry routes are Login and SignUp. Signup optionally uploads an avatar straight to Cloudinary with an unsigned preset, creates the account with Firebase Auth, upserts a

`users/<email>` document in Firestore, and stores an `auth-info` object in `localStorage` that the app uses as its session. There's no route-guard component — each page checks that `localStorage` flag on mount and redirects. I know that's UX-only security; the real gate would be Firestore rules.

**Home** is the hub. On mount it opens a Socket.IO connection and emits `join`; the server keeps an in-memory list of online users and broadcasts it to everyone, so the online list is live. It also loads my friend graph and groups from Firestore. Friends are mirrored arrays on the user documents — accepting a request adds each user's email to the other's `friends` array. I relay friend requests over sockets too, but I filter broadcasts by the recipient's email so it behaves like a targeted message, and the durable Firestore write means offline users still get the request on next login.

**Chat:** opening a room navigates to `/chat/:roomId`. The room id is a nanoid for one-to-one, or the group id for groups, so a whole group shares one room. Messages are plain objects with an id, text, sender, type, viewed flag, and timestamp. Sending emits `send-message` to the room and appends optimistically; the server relays only to that room; the peer dedupes by id. I cache messages in `localStorage` on every change and write one-to-one history back to Firestore on unmount, only if the length changed — that batches writes. Typing indicators and read receipts ride the same socket. There's also per-chat theming persisted locally.

**Video call:** the `/vc` route is WebRTC — offer, answer, and ICE candidates exchanged through the Socket.IO server as the signaling channel, with `getUserMedia` for the streams. I'll be honest about its limitation: the signaling is broadcast to all sockets rather than scoped to the room, so it really only works with two people online, and the TURN server is a placeholder.

**Cross-cutting:** it's a PWA via vite-plugin-pwa with an install prompt and an FCM background worker; config comes from Vite env vars with a hosted fallback; images go to Cloudinary; and the server is deployed on Render, the client on Vercel. If I were to keep building it, my priorities would be Firestore security rules, room-scoped video signaling with a real TURN server, moving presence to Redis so the server can scale, and breaking up the 1,900-line Home component."

## 11. Glossary

- **auth-info** — the JSON object in `localStorage` holding `{displayName, userId, email, profilePicUrl, isAuth}`; the app's de-facto session. Read via `useGetUserInfo`.
- **roomId** — identifier for a chat: a `nanoid` for 1:1 chats, or the group's id for groups. Used for Socket.IO rooms and `localStorage` keys.
- **chats/** — Firestore doc holding a 1:1 conversation; key = the two participants' display names sorted and joined with `_` (`generateId`).
- **onlineUsers** — server-side in-memory array of connected users `{id(socketId), name, email, profilePicUrl, status, isOnline}`.
- **rooms** — server-side in-memory array `{id, users[]}` tracking who's in each Socket.IO room.
- **offers** — server-side in-memory array of stored WebRTC SDP offers, replayed to new sockets.
- **firstUser** — server-side socket id of the first-connected client, used in the (fragile) `ready` handshake for video calls.
- **friendRequests / sentRequests / friends** — array fields on `users/<email>`: incoming pending, outgoing pending, and accepted friend emails.
- **members[]** — array of member emails on a `groups/<id>` document; queried with `array-contains`.
- **viewed** — boolean on a message; drives the blue double-check read receipt.
- **type** — `"text"` or `"sticker"` on a message; stickers render larger.

- **isOnline** — availability string `"online"` / `"offline"` (not a boolean), broadcast via `join`.
- **join** — socket event a client emits to announce/refresh its presence.
- **send-message / recieve-message** — socket events for chat delivery (note the misspelling of "receive" is in the actual code).
- **IsSenderTyping** — socket event carrying the typing boolean to the other participant.
- **requestJoin** — socket event pinging a specific user that someone opened a chat with them.
- **generateToken** — `firebase.js` helper that requests notification permission and fetches an FCM token.
- **SOCKET\_URL** — resolved backend URL ( `VITE_SOCKET_URL` or the Render fallback), from `lib/config.js`.
- **ml\_default** — the unsigned Cloudinary upload preset used for all image uploads.
- **cn()** — `lib/utls.js` helper merging class names via `clsx` + `tailwind-merge`.
- **PWA** — Progressive Web App; installable, service-worker-backed web app.
- **STUN/TURN** — WebRTC NAT-traversal servers; STUN discovers your public address, TURN relays media when direct connection fails.

## Open questions / things I couldn't verify

These are places where the code is unclear, unused, inconsistent, or looks buggy — flagged rather than guessed.

1. **Video call is not room-scoped.** `Chat.jsx:563` passes `{userData, roomId}` to `/vc`, but `video-call.jsx` never reads `useLocation` / `state`. Signaling uses `socket.broadcast.emit(...)` (`video-call.jsx:268,292,331`) and the server replays `offers[0]` to every new socket (`server.js:170-172`). This only works correctly with exactly two people online. **Could not verify** it ever worked for 3+ users; it appears broken by design.
2. **addRegisteredUser / registered/users\_list collection is dead code.** Defined in `useFirestore.js:61-96` but never called; Home uses `getRegisteredUsers` (the `users` collection) instead.
3. **handleDeleteGroup never deletes from Firestore.** `Home.jsx:650-653` emits a socket event and filters local state but doesn't call the Firestore `deleteGroup` (`useFirestore.js:135-141`). Also, I couldn't find any UI button wired to `handleDeleteGroup`, so group deletion may be unreachable in the current UI.
4. **handleViewMessages is unused.** Defined in `Chat.jsx:226-242` to mark received messages as viewed, but its only call site is commented out (`Chat.jsx:410`). So incoming messages aren't marked `viewed` after render — read receipts only reflect the room-occupancy check at send time.
5. **Profile edits don't reach Firestore.** `handleSaveProfile` (`Home.jsx:523-579`) updates `localStorage` and re-emits `join`, but never writes the new `displayName`/`bio`/`pic` to the `users/<email>` doc; `bio` isn't stored in Firestore at all. Durable profile can drift from the local one.
6. **Timestamp type inconsistency.** Text messages use `new Date().toISOString()` (string) (`Chat.jsx:466`) while stickers use `new Date()` (Date object) (`Chat.jsx:497`). Rendering wraps both in `new Date(...)` so it usually works, but the stored shapes differ.
7. **Message id = Date.now()** can collide within the same millisecond, which would make the dedupe check drop a legitimate message.
8. **Unused dependencies (likely).** `@chakra-ui/react`, `next-themes`, and the `cloudinary` SDK are in `client/package.json` but I found no imports of them in `src/` (uploads use raw `axios`, theming uses the custom context). `nanoid` is imported in `server.js:6` but not used server-side. **Could not fully verify** across every file, but `grep`/reads didn't surface usages.
9. **SplitText.jsx presence.** It exists alongside `BlurText.jsx`; only `BlurText` is imported in `Home.jsx`. I didn't find an import of `SplitText`, so it may be unused. (Not exhaustively verified.)

10. `Login.jsx` **Google loading state** uses a fixed `setTimeout(() => setLoading(false), 2000)` (`Login.jsx:102`) decoupled from the actual async result, so the spinner timing is cosmetic and can mismatch reality.
11. **Committed credentials.** Firebase config/API key, Cloudinary cloud name + preset, and the FCM VAPID key are hardcoded in source. The Firebase apiKey is a public client identifier (normal to ship), but committing all of this is a practice smell; real protection depends on Firestore rules that aren't in this repo.
12. `get-room-info / update-room-info` **reliability.** Presence in a room depends on `onlineUsers` still containing the socket; the sender detection in `Chat.jsx:405-414` picks "the other user in the room," which assumes a 2-person room and won't identify a specific peer in a group.